

ANEXO II. EXPERIMENTACIÓN

1. Algoritmos utilizados

Durante la fase de experimentación utilizaremos 7 modelos con mecanismos de aprendizaje distintos. Para cada combinación de escenario y estrategia, los 7 modelos evalúan sobre el mismo conjunto de entrenamiento y prueba, permitiendo identificar qué algoritmo se adapta mejor a cada contexto diagnóstico.

- Regresión Logística: modelo lineal que estima la probabilidad de PDAC en función de los biomarcadores. Si modelos más complejos no superan significativamente a la regresión logística, indicaría que el problema es linealmente separable y no justifica complejidad adicional.
- Random Forest: construye múltiples árboles de decisión independientes sobre subconjuntos aleatorios de datos y features, y promedia sus predicciones. Se incluye por su robustez ante outliers extremos como los que presentan CA19-9 y TFF1, y porque maneja bien las relaciones no lineales entre biomarcadores.
- SVM (Support Vector Machine): busca el hiperplano que maximiza el margen de separación entre clases en un espacio de alta dimensión. Efectivo especialmente cuando las clases no son linealmente separables, como ocurre en PDAC vs Benigno donde los biomarcadores se solapan considerablemente.
- KNN (K-Nearest Neighbors): clasifica cada paciente según la clase mayoritaria de sus 7 vecinos más cercanos en el espacio de features. Su rendimiento indica si los grupos están bien separados en el espacio de biomarcadores.
- XGBoost: construye árboles secuencialmente donde cada uno corrige los errores del anterior, con regularización matemática para evitar generalizaciones sobre datos nuevos. Se incluye como modelo por su buen rendimiento en problemas de clasificación con datos tabulares clínicos, como en nuestro caso: un número moderado de muestras, variables numéricas continuas con distribuciones asimétricas y relaciones no lineales entre biomarcadores.
- Gradient Boosting: mismo principio que XGBoost pero con la implementación clásica de sklearn. Se incluye como alternativa para verificar si las diferencias de rendimiento entre implementaciones de boosting son relevantes en este dataset.
- AdaBoost: boosting adaptativo que en cada iteración aumenta el peso de los pacientes mal clasificados, obligando al siguiente árbol a enfocarse en los casos más difíciles. Es relevante para detectar los casos límite entre grupos, como a PDAC en estadios iniciales con biomarcadores bajos.

Para determinar el modelo de diagnóstico óptimo, usaremos el ROC-AUC (Area Under the Receiver Operating Characteristic Curve). Este es el área bajo la curva ROC, que representa gráficamente la relación entre la tasa de verdaderos positivos (sensibilidad) y la tasa de falsos positivos (especificidad) para todos los umbrales de decisión posibles.

Este se calcula mediante verdaderos positivos (TP, pacientes con PDAC que el modelo clasifica correctamente), falsos positivos (FP, pacientes que el modelo clasifica erróneamente como PDAC), verdaderos negativos (TN, pacientes sanos que el modelo clasifica correctamente como no PDAC) Y falsos negativos (FN, pacientes con PDAC que el modelo clasifica erróneamente como sanos).

Un valor de 1 indica un clasificador perfecto que distingue todos los PDAC, mientras que 0.5 equivale a una clasificación aleatoria sin poder discriminativo, es por ello por lo que consideraremos un valor de ROC-AUC=0.7 y un 0.5 de sensibilidad los mínimos para considerar un modelo útil. Es por ello que el modelo con mayor ROC-AUC en cada combinación se seleccionará como representante en la tabla comparativa final y determinando el modelo con mayor capacidad de priorizar correctamente los pacientes de mayor riesgo para pruebas diagnósticas.

2. Herramientas y librerías utilizadas

La experimentación se realiza en Python, un lenguaje de programación ampliamente utilizado en el ámbito del análisis de datos y el Machine Learning. Además, se usan las siguientes librerías:

- Numpy: utilizada para computación matemática y procesamiento de datos. Permite el manejo de arrays multidimensionales y matrices de gran tamaño, y proporciona multitud de funciones matemáticas para operar sobre estas [47]. En el contexto de este proyecto enfocamos su uso en operaciones numéricas, cálculo de estadísticas y manipulación de arrays de datos.
- Pandas: análisis y manipulación de datos que proporciona estructuras de datos de alto nivel, en concreto DataFrame, que permite trabajar con datos tabulares fácilmente [48]. En este proyecto se utiliza para la carga del dataset, selección y eliminación de columnas, análisis estadístico descriptivo y generación de tablas cruzadas.
- Matplotlib: proporciona una interfaz para la creación de gráficos 2D de publicación científica [49]. La utilizamos para la generación de gráficos del análisis exploratorio y evaluación de modelos.
- Seaborn: construida sobre Matplotlib, proporciona una interfaz de alto nivel para la creación de gráficos estadísticos [50]. En este contexto se utiliza para la generación de heatmaps, matrices de correlación y gráficos comparativos.
- Scikit-learn: es la librería de referencia de Machine Learning en Python, integrando gran variedad de algoritmos de aprendizaje supervisado y no supervisado [51]. En nuestro caso se emplea para el preprocesamiento de datos, entrenamiento, validación y evaluación de los modelos.
- Scipy: proporciona algoritmos para optimización, álgebra lineal, integración y estadística [52]. En este proyecto se utiliza específicamente para el análisis de clustering jerárquico.

3. Código relevante

los bloques de código más relevantes del notebook de experimentación, organizados siguiendo el mismo orden que el apartado 5 de la memoria. El código completo está disponible en el repositorio de GitHub y en la plataforma Kaggle.

VI.1 Filtrado y preprocesamiento

Filtro de estadios iniciales, normalización por creatinina, codificación de variables y separación de la copia para ML antes de la imputación.

```
# Filtro de estadios: solo PDAC en estadios I-II + todos los no-PDAC
pdac_iniciales = (datos['diagnosis'] == 3) & (
    datos['stage'].isin(['I', 'IA', 'IB', 'II', 'IIA', 'IIB']))
no_pdac = datos['diagnosis'].isin([1, 2])
datos = datos[pdac_iniciales | no_pdac].copy()

# Normalización por creatinina
for bio in ['LYVE1', 'REG1B', 'TFF1', 'REG1A']:
    datos[bio + '_normalizado'] = datos[bio] / datos['creatinine']

# Codificación numérica de sexo (LabelEncoder: F=0, M=1)
datos['sex'] = LabelEncoder().fit_transform(datos['sex'])

# Copia para ML ANTES de imputar (evita data leakage)
datos_ml = datos.copy()

# Imputación por mediana solo para el EDA
datos['plasma_CA19_9'] = datos['plasma_CA19_9'].fillna(
    datos['plasma_CA19_9'].median())
datos['REG1A_normalizado'] = datos['REG1A_normalizado'].fillna(
    datos['REG1A_normalizado'].median())
```

Código VI.1. Filtrado y preprocesamiento del dataset.

VI.2 Función de entrenamiento y evaluación

El pipeline de imputación y estandarización se crea dentro de la función para que cada combinación aprenda sus propias estadísticas exclusivamente de X_train.

```
todos_resultados, todos_modelos = {}, {}

def entrenar_evaluar(X_train, X_test, y_train, y_test):
    pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])
    X_train_imp = pipeline.fit_transform(X_train)
    X_test_imp = pipeline.transform(X_test)

    resultados, modelos_entrenados = {}, {}
```

```

for i, modelo in modelos_ML.items():
    m = copy.deepcopy(modelo)
    m.fit(X_train_imp, y_train)
    probabilidad = m.predict_proba(X_test_imp)[: , 1]
    prediccion = m.predict(X_test_imp)
    tn, fp, fn, tp = confusion_matrix(y_test, prediccion).ravel()

    resultados[i] = {
        'ROC-AUC': round(roc_auc_score(y_test, probabilidad), 3),
        'Bal.Accuracy': round(balanced_accuracy_score(y_test, prediccion), 3),
        'Sensibilidad': round(tp/(tp+fn) if (tp+fn)>0 else 0, 3),
        'Especificidad': round(tn/(tn+fp) if (tn+fp)>0 else 0, 3),
        'prob': probabilidad,
        'pred': prediccion,
    }
    modelos_entrenados[i] = (m, X_train_imp, X_test_imp, pipeline)

return resultados, modelos_entrenados

```

Código VI.2. Función de entrenamiento y evaluación de modelos.

VI.3 Bucle principal 3×3

Recorre las 9 combinaciones de escenario y estrategia, filtra el subconjunto correspondiente y llama a entrenar_evaluar.

```

for escenario, diagnosis in ESCENARIOS.items():
    subset = datos_ml[datos_ml['diagnosis'].isin(diagnosis)].copy()
    subset['PDAC'] = (subset['diagnosis'] == 3).astype(int)

    for estrategia, features in ESTRATEGIAS.items():
        X, y = subset[features], subset['PDAC']

        X_tr, X_te, y_tr, y_te = train_test_split(
            X, y, test_size=0.20, stratify=y, random_state=SEED)

        resultados, modelos_entrenados = entrenar_evaluar(
            X_tr, X_te, y_tr, y_te)

        todos_resultados[escenario, estrategia] = resultados
        todos_modelos[escenario, estrategia] = (
            modelos_entrenados, X_tr, X_te, y_tr, y_te, features)

    mejor = max(resultados, key=lambda n: resultados[n]['ROC-AUC'])
    print(escenario, estrategia, '-> Mejor:', mejor,
          'ROC-AUC:', round(resultados[mejor]['ROC-AUC'], 3),
          'Sens:', round(resultados[mejor]['Sensibilidad'], 3))

```

Código VI.3. Bucle principal de entrenamiento.

VI.4 Recuperación del mejor modelo

Identifica la mejor combinación global y recupera el modelo entrenado sin necesidad de reentrenar.

```
# Mejor combinación por ROC-AUC global
mejor_ROC_AUC = max(todos_resultados,
                    key=lambda k: max(v['ROC-AUC'] for v in todos_resultados[k].values()))

mejor_modelo = max(todos_resultados[mejor_ROC_AUC],
                  key=lambda n: todos_resultados[mejor_ROC_AUC][n]['ROC-AUC'])

# Recuperar datos y modelo ya entrenados
modelos_entrenados, X_tr, X_te, y_tr, y_te, features = \
    todos_modelos[mejor_ROC_AUC]
modelo_final, X_train_imp, X_test_imp, _ = \
    modelos_entrenados[mejor_modelo]

prob_final = modelo_final.predict_proba(X_test_imp)[: , 1]
pred_final = modelo_final.predict(X_test_imp)

# Resultado: ('E1','S3') | XGBoost | ROC-AUC=0.973
print(classification_report(y_te, pred_final,
                           target_names=['no-PDAC', 'PDAC']))
```

Código VI.4. Recuperación e informe del mejor modelo.

VI.5 Análisis por umbral de decisión

Evalúa el impacto clínico de distintos umbrales sobre sensibilidad, especificidad y número de falsos negativos y positivos.

```
print(f"{'Umbral':<8}{ 'Sensibilidad':<15}{ 'Especificidad':<15}"
      f"{'Precisión':<12}{ 'FN':<5}{ 'FP'}")

for i in [0.20, 0.50, 0.70]:
    yp = (prob_final >= i).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_te, yp).ravel()
    sens = tp/(tp+fn) if (tp+fn)>0 else 0
    spec = tn/(tn+fp) if (tn+fp)>0 else 0
    prec = tp/(tp+fp) if (tp+fp)>0 else 0
    print(f"{'i':<8}{round(sens,3):<15}{round(spec,3):<15}"
          f"{'round(prec,3):<12}{fn:<5}{fp}")
```

Código VI.5. Tabla de métricas por umbral de decisión.

VI.6 Interpretabilidad SHAP

Cálculo de valores SHAP y generación del diagrama de puntos y el ranking de importancia media.

```
explainer = shap.TreeExplainer(modelo_final)
shap_values = explainer.shap_values(X_test_imp)
```

```

# Compatibilidad entre versiones de SHAP
if isinstance(shap_values, list):
    sv = shap_values[1]
elif hasattr(shap_values, 'values'):
    sv = shap_values.values
    if sv.ndim == 3: sv = sv[:, :, 1]
else:
    sv = shap_values
    if sv.ndim == 3: sv = sv[:, :, 1]

# Diagrama de puntos: dirección e importancia por paciente
shap.summary_plot(sv, X_test_imp, feature_names=features, show=False)
plt.title(f'Dirección del diagnóstico\n{mejor_modelo} | {mejor_ROC_AUC}')
plt.show()

# Ranking de importancia media
mean_shap = np.abs(sv).mean(axis=0)
shap_df = pd.DataFrame({'Biomarcador': features, 'Media': mean_shap})
print(shap_df.sort_values('Media', ascending=False).to_string(index=False))

```

Código VI.6. Interpretabilidad SHAP.

VI.7 Interpretabilidad DALEX

Importancia por permutación, perfiles de dependencia parcial y explicaciones locales para dos pacientes representativos.

```

explainer = dx.Explainer(
    modelo_final,
    pd.DataFrame(X_test_imp, columns=features),
    y_te, label='XGBoost E1-S3')

# Importancia global: caída en ROC-AUC al permutar cada variable
explainer.model_parts(loss_function='1-auc').plot()

# Perfiles de dependencia parcial
explainer.model_profile(variables=features).plot()

# Explicación local — paciente con mayor probabilidad de PDAC
idx = prob_final.argmax()
paciente = pd.DataFrame(X_test_imp[idx:idx+1], columns=features)
explainer.predict_parts(paciente, type='break_down').plot(
    title=f'Paciente PDAC (prob={prob_final[idx]:.3f})')

# Explicación local — paciente con mayor certeza de no-PDAC
idx = prob_final.argmin()
paciente = pd.DataFrame(X_test_imp[idx:idx+1], columns=features)
explainer.predict_parts(paciente, type='break_down').plot(
    title=f'Paciente no-PDAC (prob={prob_final[idx]:.3f})')

```

Código VI.7. Interpretabilidad DALEX.